

DEMAND FORECASTING USING PySpark

Distributed Data Analysis Project Report

Date: 2 May 2026

1. Executive Summary

This project presents a scalable, distributed data architecture built to predict weekly commerce product demand. By engineering a multi-stage PySpark pipeline over a MinIO-backed Medallion Architecture, the system successfully processes and transforms over 913,000 rows of historical sales data from raw ingestion (Bronze) to analytics-ready features (Gold).

The analytical phase benchmarks five forecasting approaches against a chronological train/test split. This includes Spark MLlib regressors (Linear Regression, Random Forest, GBT) alongside dedicated time-series models (ARIMA, Prophet). All resulting models and metrics are saved back to the Gold layer to guarantee full reproducibility.

To bridge the gap between data engineering and end-user application, the final Prophet model is served through an interactive Streamlit dashboard. This application provides real-time forecasting, dynamic filtering, and a built-in MLOps suite designed to detect data anomalies and monitor model drift.

Key Model Performance Outcomes

Model	RMSE	MAE	MAPE	R ²
Linear Regression	38.42	25.79	7.29%	0.9656
Random Forest	34.00	23.20	6.26%	0.9730
Gradient Boosting	35.34	24.73	6.78%	0.9709
ARIMA	289.63	248.71	57.59%	-0.96
Prophet	31.0764	23.2373	6.43%	0.9775

NOTE Spark MLlib models and Prophet significantly outperform the ARIMA model. While both ARIMA and Prophet fit independently per store/item pair, only ARIMA struggled without the cross-series features (store/item OHE, lag/rolling stats) leveraged by the MLlib models.

2. Technology Stack & Dependencies

Core Technologies

Layer	Technology	Version / Notes
Language	Python	3.13
Distributed Processing	Apache Spark (PySpark)	3.5.4 — local[*] mode
Object Storage	MinIO	S3-compatible data lake; native binary and Docker
JVM Runtime	Java JDK	17 (required by PySpark)
Serialization	Apache Parquet	Snappy compression via Spark
S3A Connector	hadoop-aws	3.3.4 (matched to PySpark's bundled Hadoop)

3. Key Components & Modules

3.1 Data Lake Architecture

The project implements a three-tier Bronze → Silver → Gold within a single MinIO bucket (Data-lake). Each zone represents a progressively refined version of the data, from raw ingestion through to model-ready feature sets.

Zone	Content	Partitioning	Parts	Rows
Bronze	Raw CSV as Parquet	—	—	913,000
Silver / daily	Cleaned Data	By store (10)	10	913,000
Silver / weekly	Weekly aggregated	By store (10)	10	131,000
Gold / features	Full feature set (66+ columns)	By store (10)	10	105,000
Gold / models	5 trained model artifacts	—	—	—
Gold / metrics	Model Comparison CSV	—	—	5 rows

3.2 Ingestion Module

- First Reads raw data csv and runs 6 validation checks: row count ,null counts, non-negative sales, store range [1,10], item range [1,50], and date range [2013–2017].
- Then writes the validated DataFrame to s3a as Parquet in Bronze Layer with mode('overwrite') for idempotency.

3.3 Preprocessing Module

- Casts date to DateType, drops duplicates, removes negative-sales rows
- Loads the US Holiday Dates CSV (343 entries), drop duplicates to be unique dates
- Writes Silver/daily layer partitioned by store
- Then Extracts year and week_of_year from date
- Groups by (store, item, year, week_of_year) to compute weekly_sales = SUM(sales)
- and week_has_holiday = MAX(is_holiday)
- Writes Silver/weekly layer partitioned by store almost 131K rows

3.4 Feature Engineering

A consolidated module that transforms the Silver/weekly layer into the Gold/features layer through 7 sequential steps, producing a final schema of 18 columns with 105K rows and a 68-dimensional feature vector for Spark MLlib.

Step	Type	Features Added	Technique
①	Temporal	month, quarter, is_year_end, abs_week	Date arithmetic from year + week
②	Lag	lag_1_week, lag_4_week, lag_52_week	Window partition by(store, item) then order by(abs_week) with F.lag()
③	Rolling	rolling_4_week_avg, rolling_12_week_avg	Window rows between(-N, -1) with F.avg()
④	Null Drop	—	Drops first 52 weeks per series (warmup period)
⑤	One-hot encoding (OHE)	Store ohe (9-dim), item ohe (49-dim)	String Indexer > One Hot Encoder pipeline (drop-last)
⑥	Column Select	—	Orders 18 columns into clean gold schema
⑦	Write	—	Writes to gold/features/ partitioned by store

3.5 Model Implementations

Five forecasting models are implemented and evaluated. Three Spark MLlib regressors leverage the full 68-dimensional cross-series feature vector, while ARIMA and Prophet fit independently per store/item pair (500 total) in parallel via joblib.

Training Orchestration

- Load & Split: Chronological split — train (year ≤ 2016, ~104K rows), test (year = 2017, ~26K rows)
- Train: All 5 models sequentially with timing instrumentation
- Persist: Spark models via .save(), ARIMA/Prophet via pickle to MinIO
- Evaluate: Compute RMSE/MAE/MAPE/R² for each model and save metrics
- Cache Management: cache()/unpersist() for faster train/test DataFrames across all Spark models

3.6 Evaluation Module

Dual-path metric computation handles both Spark and Pandas prediction outputs. Computed metrics are RMSE, MAE, MAPE and R².

- compute_spark_metrics(): For Spark MLlib predictions (collects to Pandas for calculation)
- compute_pandas_metrics(): For ARIMA/Prophet predictions (already in Pandas)
- save_metrics(): Writes to both results Into metric gold layer

3.7 Pipeline & MLOps

- Full pipeline :End-to-end orchestration with 7 stages (ingest → bronze → silver_daily → silver_weekly → gold → train → evaluate). Supports --stage argument to resume from any stage.
- Minio setup: Subprocess-based pipeline runner that calls each module as python -m src.xxx with --continue-on-error support.
- Monitor : Production monitoring — alerts if current RMSE exceeds training baseline by >10%; alerts if any column exceeds 1% null rate joins predictions with ground-truth from MinIO Silver to calculate live RMSE.

3.8 Streamlit App

Interactive web application serving as the production interface for demand forecasting and operational monitoring.

Page	Features
Home	CSV upload, data preview, 'Run Prophet Forecast' button
Year Predictions	Filtered predictions table, CSV download, integrated monitoring suite
Visualization	Charts — weekly sales by store/item, total predicted by store/item
Team	Team member with roles and IDs

Key Implementation Details

- Loads production Prophet models from MinIO
- Reuses the exact same preprocessing pipeline cleaning and aggregate to weekly for uploaded data
- Integrated monitoring: it checks the performance and check missing data within uploaded data

3.9 Data Flow Overview

The end-to-end data flows through six stages from raw CSV ingestion through to the production Streamlit dashboard:

- **Stage 1:** (Ingest): raw_data.csv (913K rows) + US Holiday Dates CSV are validated and uploaded to the Bronze layer as Parquet.
- **Stage 2:** (Clean): Remove duplicates, filters negatives, and enriches with the holiday flag. Output stored in Silver/daily.
- **Stage 3:** (Aggregate): groups daily records to weekly (store, item, year, week). Output stored in Silver/weekly (131K rows).
- **Stage 4:** (Feature Engineering): adds temporal, lag, rolling, and OHE features. Output stored in Gold/features (105K rows, 68-dim vectors).
- **Stage 5:** (Train & Evaluate): trains all 5 models and persists them to Gold/models. Metrics saved to Gold/metrics.
- **Stage 6:** (Serve): loads Prophet models from MinIO and serves interactive forecasts; monitor.py performs drift and data quality checks.

Exploratory Data Analysis Visualizations

The following charts highlight key patterns, correlations, and distributions within the dataset, serving as the factual foundation for our model design decisions.

Figure 1 — Distribution of Weekly Sales

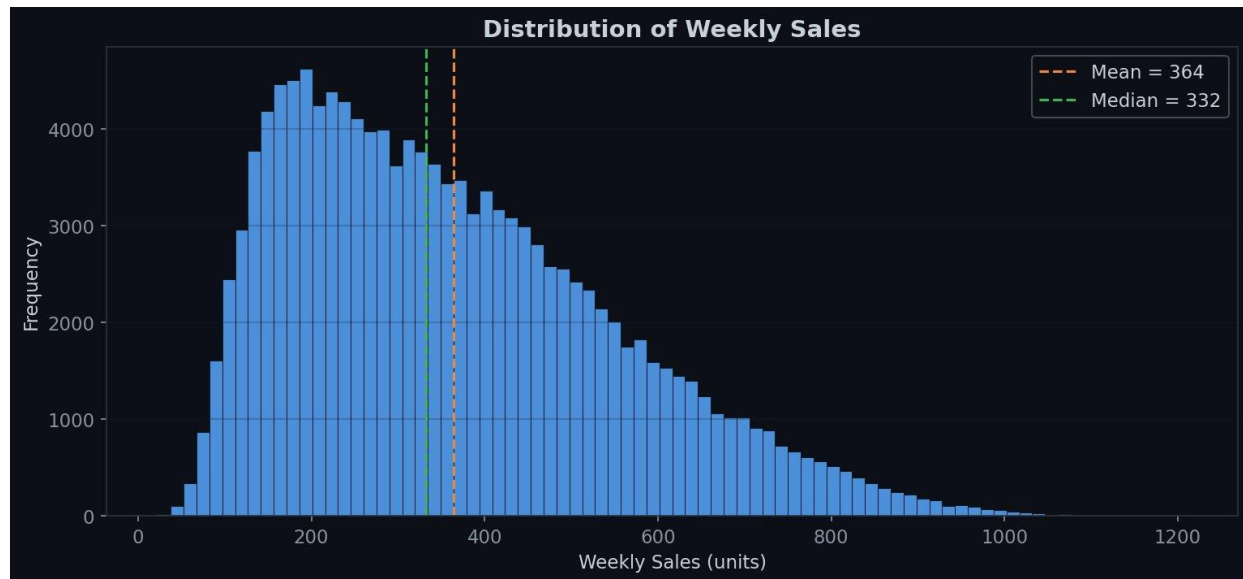


Figure 1: Histogram of weekly sales volumes across all stores and items (2013–2017).

The distribution exhibits a right-skewed pattern, with the majority of weekly sales concentrated between 100 and 400 units. The median (332 units) falls noticeably below the mean (364 units), confirming the positive skew driven by a long tail of high-volume weeks reaching up to 1,200 units. This skewness suggests that most store–item combinations record moderate weekly sales, while a smaller subset of high-demand products significantly pulls the average upward — an important consideration when selecting and evaluating forecasting models.

Figure 2 — Total Weekly Sales Over Time



Figure 2: Total weekly units sold across all stores and items, 2013–2017.

The line chart reveals a strong and consistent annual seasonal cycle repeated across all five years. Sales ramp up from a low base at the start of each year, peak sharply around weeks 26–30 (mid-summer), then decline into the year-end period. A notable sharp dip appears at the boundary between years (week 52–1), likely reflecting a data gap or reset.

artifact or inventory reset. Overall volume grows year-over-year from approximately 100K–200K units in 2013 to peaks exceeding 260K in 2017, indicating steady demand growth across the retailer’s catalog. This recurring seasonal pattern directly motivates the inclusion of temporal and lag features in the feature engineering pipeline.

Figure 3 — Monthly Seasonality: Weekly Sales Distribution

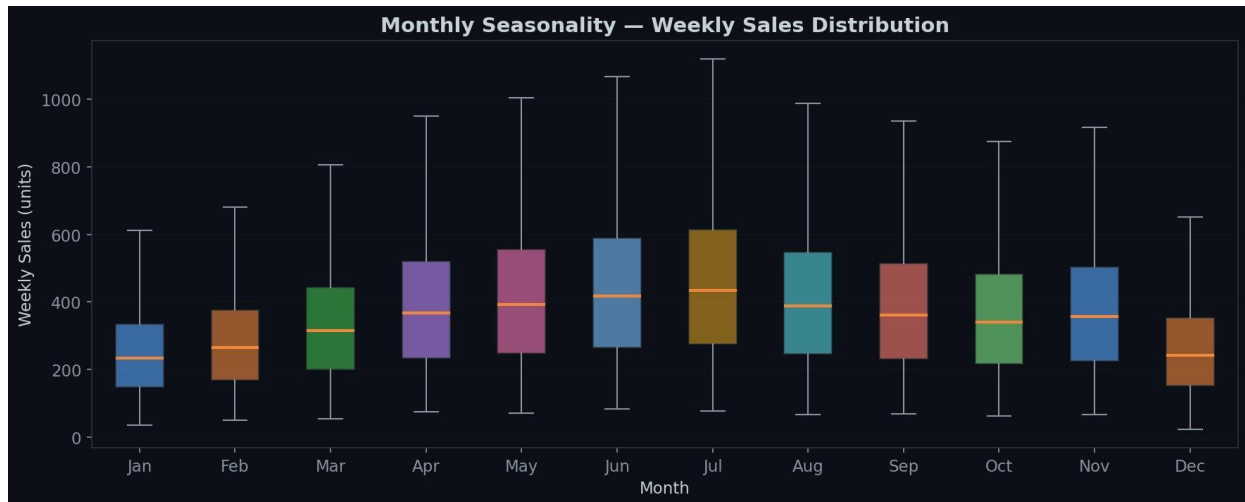


Figure 3: Box plot of weekly sales distributions grouped by month, aggregated across all years, stores, and items.

The box plot breaks down weekly sales variability by calendar month, exposing a clear bimodal seasonal structure. Sales are weakest in January, February, and December (medians around 240–270 units), reflecting post-holiday and winter slowdowns. Demand builds steadily through spring, peaking in June and July with medians near 415–445 units and the widest interquartile ranges — indicating both higher average volumes and greater store-to-store variability during the summer peak. The tail whiskers for summer months extend well above 1,000 units, highlighting occasional extreme demand events. This monthly pattern confirms that the month and quarter features engineered in the Gold layer are meaningful predictors for the Spark MLlib models.

Figure 4 — Average Weekly Sales by Week-of-Year Heatmap

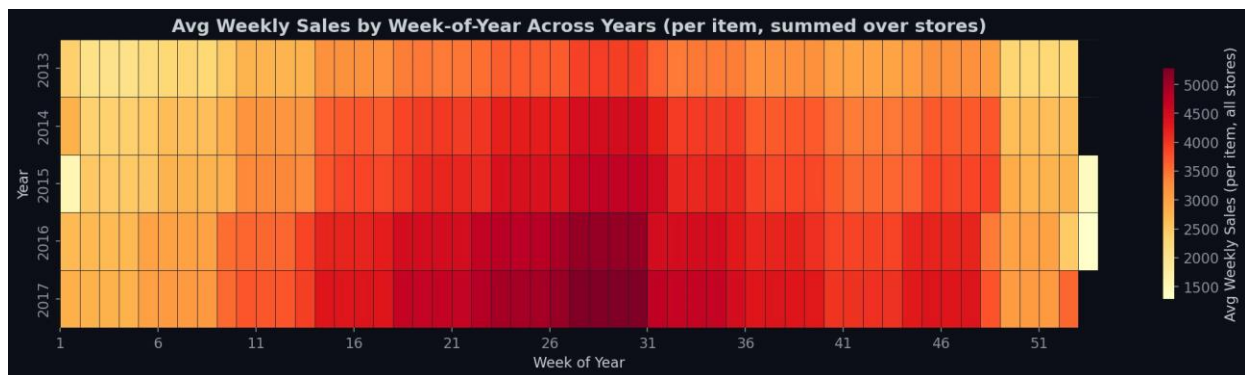


Figure 4: Heatmap of average weekly sales per item (summed over stores) by week-of-year and year (2013–2017).

The heatmap provides a two-dimensional view of sales intensity across both the week-of-year axis and the year axis. The deep red band consistently spanning weeks 21–32 (approximately May through August) across all years confirms the mid-year sales peak observed in prior charts, with average values exceeding 4,500–5,000 units per item. The cooler yellow-orange tones at the start and end of each year (weeks 1–5 and 50–53) reflect the consistent low-demand periods. Notably, sales intensity deepens from 2013 through 2016 (darker red), corroborating the year-over-year growth trend, while 2017 appears slightly lighter — consistent with it being the

held-out test year with a partial record. The `abs_week` feature in the pipeline encodes this cross-year ordering, enabling lag features to capture this multi-year growth pattern.

Figure 5 — Total Sales Per Store (2013–2017)

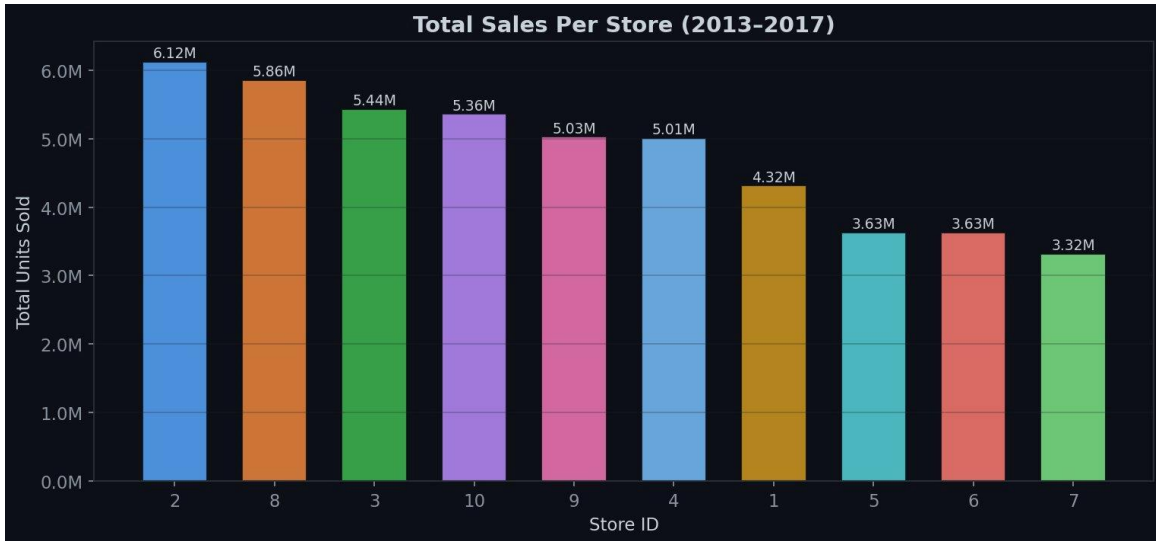


Figure 5: Ranked bar chart of total units sold per store across the full dataset (2013–2017), sorted by descending volume.

The bar chart reveals significant volume disparity across the 10 stores. Store 2 leads with 6.12M total units sold, nearly double the volume of Store 7 at the bottom (3.32M). The top four stores (2, 8, 3, 10) each exceeded 5M units, while the bottom three (1, 5, 6, 7) cluster between 3.3M and 4.3M. This ~85% spread between the highest and lowest performing stores underscores the importance of store-level identity in the forecasting models. The one-hot encoding of the store feature (9-dimensional OHE vector in the Gold layer) directly captures these per-store baseline differences, allowing the Random Forest and Gradient Boosting models to learn store-specific demand levels.

Figure 6 — Average Weekly Sales Per Store Over Time

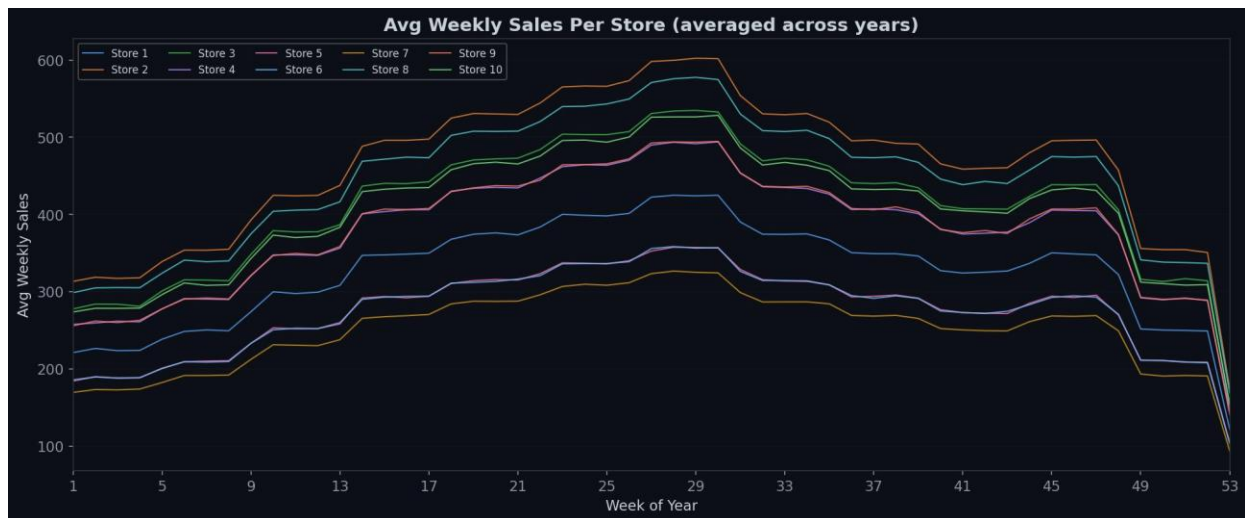


Figure 6: Average weekly sales per store (averaged across all years) plotted against week of year.

The multi-line chart shows that all 10 stores share the same seasonal shape — a gradual rise from week 1, a broad plateau peaking around weeks 25–33 (summer), followed by a steep drop toward week 53 — but are vertically stratified by their individual volume baselines. Stores 2 and 8 consistently occupy the top band (peaking near 580–

605 units/week), while Stores 4 and 7 form the lower band (peaking around 260–300 units/week). The parallel shape of all lines confirms that seasonality is a store-invariant signal, validating the use of shared temporal features (week_of_year, month, quarter) across all store-item series rather than requiring separate seasonal models per store. The sharp convergence of all lines at week 53 reflects the year-boundary dip observed in the aggregate time-series chart.

Figure 7 — Average Daily Purchasing Intensity Heatmap

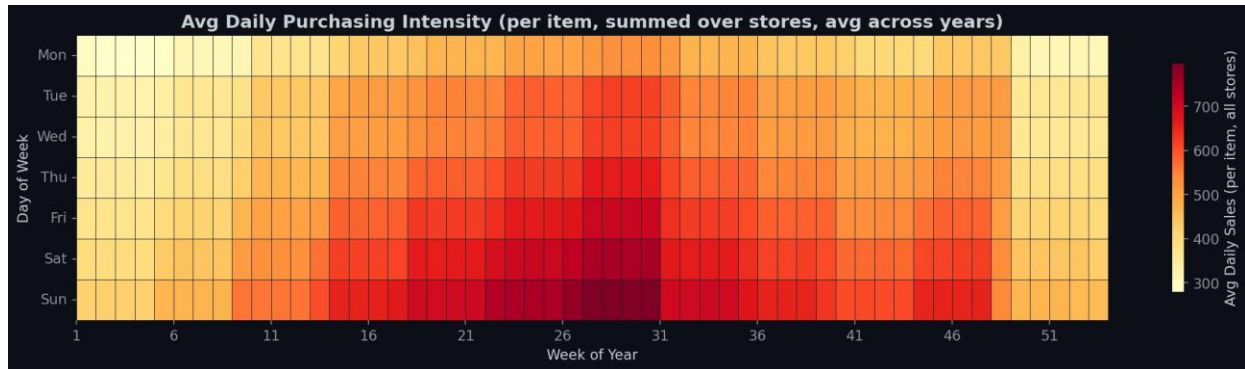


Figure 7: Heatmap of average daily purchasing intensity per item (summed over stores, averaged across years) by day of week and week of year.

This heatmap adds a daily dimension to the seasonal analysis by cross-referencing the week of year with the day of week. The darkest red cells (values approaching 700+ units) are concentrated in the Thursday–Sunday band during weeks 21–33, confirming that weekend and late-week shopping is most intense during the summer peak. Monday consistently registers the lightest intensity across all weeks, suggesting a systematic day-of-week effect on purchasing behavior. Although the pipeline aggregates data to weekly granularity (eliminating the intra-week signal), this chart validates the week_has_holiday feature: holidays frequently fall on Mondays and Fridays, redistributing daily demand within the week and making the weekly holiday flag a meaningful predictor for the Spark MLlib models.

Figure 8 — Top & Bottom 10 Items by Total Sales



Figure 8: Side-by-side horizontal bar charts showing the top 10 (green) and bottom 10 (red) items by total units sold across all stores and years.

Item-level demand varies dramatically across the 50-item catalog. The top performers — items 15 and 28 — each accumulated approximately 1.6M total units over the five-year period, while the lowest-performing item (Item 5) reached only ~0.33M units, a roughly 5× difference in cumulative volume. The top 10 items are tightly clustered between 1.4M and 1.6M units, suggesting a cohort of consistently high-demand products. The bottom 10 show more spread (0.33M–0.53M), pointing to greater variability among low-volume items. This item-level heterogeneity is encoded through the 49-dimensional item OHE vector in the Gold feature layer, enabling tree-

based models to learn item-specific demand baselines and avoid the averaging artifacts that undermine per-series ARIMA and Prophet fits.

Figure 9 — Demand Volatility by Item (Max vs Min Weekly Sales)

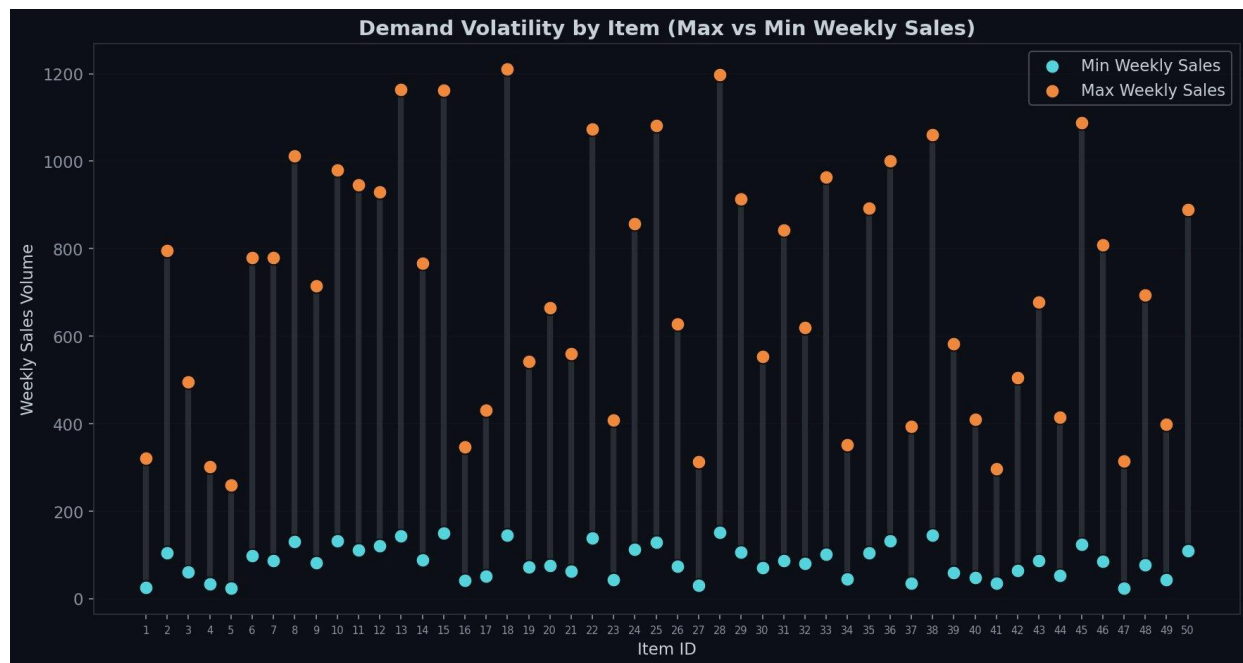


Figure 9: Lollipop chart showing the minimum (teal) and maximum (orange) weekly sales observed per item across all stores and years.

The lollipop chart exposes the intra-item demand range for all 50 products. Minimum weekly sales (teal dots) cluster near zero across most items, indicating that every product experiences near-zero demand weeks at some point. Maximum weekly sales (orange dots) vary widely, with the most volatile items (13, 15, 18, 27) reaching peaks of 1,150–1,220 units/week while the least volatile items (3, 5, 26) barely exceed 300–500 units at their peak. The tall stems on high-volatility items indicate a wide seasonal swing, making these items harder to forecast accurately and contributing disproportionately to model error. This justifies the inclusion of rolling window averages (`rolling_4_week_avg` and `rolling_12_week_avg`) as smoothing features that help the model distinguish genuine trend from noise in high-swing series.

Figure 10 — Feature Correlation Matrix**Figure 10: Lower-triangular Pearson correlation heatmap of the key numerical features in the Gold layer.**

The correlation matrix reveals two distinct feature clusters. The lag and rolling features (lag_1_week, lag_4_week, lag_52_week, rolling_4_week_avg, rolling_12_week_avg) are extremely strongly correlated with the target weekly_sales ($r = 0.94\text{--}0.98$), confirming they are the most predictive signals in the model and explaining why the Random Forest achieves an R^2 of 0.973. The temporal features (week_of_year, month, quarter) are near-perfectly correlated with each other ($r \approx 0.97\text{--}1.00$) but only weakly correlated with the target ($r = 0.08\text{--}0.09$), indicating they contribute seasonal shape information rather than direct level prediction. The week_has_holiday feature shows near-zero correlations across the board ($|-0.06|$ to $|0.07|$), suggesting it acts as a subtle binary correction signal rather than a primary driver — but its inclusion is still justified as it captures a systematic demand redistribution effect on holiday weeks.

Model Evaluation Charts

The following charts visualize the performance comparison of all five trained models across three evaluation metrics: RMSE/MAE error, R^2 explanatory power, and MAPE percentage error.

Figure 11 — Model Error Comparison (RMSE & MAE)

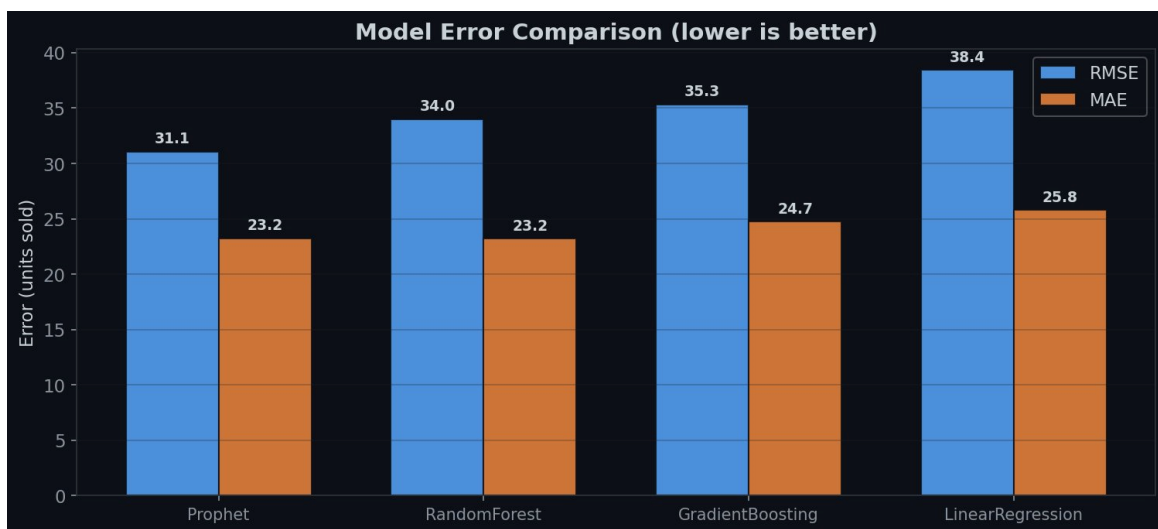


Figure 11: Grouped bar chart comparing RMSE (blue) and MAE (orange) across the four Spark MLlib and Prophet models (lower is better). Note: ARIMA is excluded from this chart due to its extreme error values (RMSE = 289.63).

Among the four models displayed, Random Forest achieves the lowest RMSE (34.0) and ties with Prophet for the lowest MAE (23.2), making it the best overall performer on absolute error metrics. Gradient Boosting follows closely (RMSE 35.3, MAE 24.7), confirming that ensemble tree methods are well-suited to this feature-rich cross-series dataset. Linear Regression trails slightly with RMSE 38.4 and MAE 25.8, still delivering a competitive result given its simplicity. Prophet's low MAE of 23.2 is notable — matching Random Forest on mean absolute error — suggesting it handles typical-magnitude errors well but is more susceptible to large outlier errors that inflate its RMSE. The narrow spread between all four models (RMSE range: 34.0–38.4) indicates that the engineered features are the dominant performance driver across architectures.

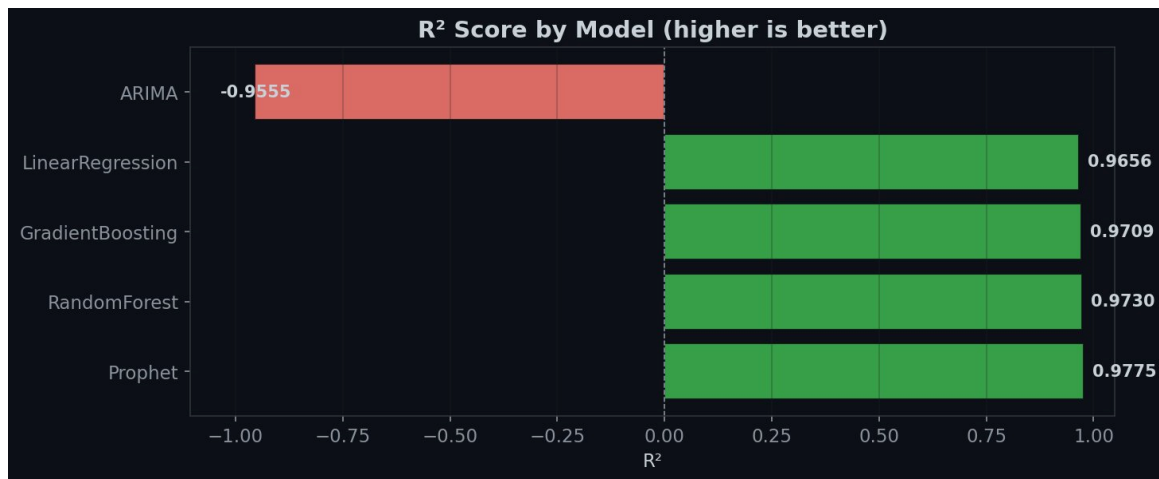
Figure 12 — R^2 Score by Model

Figure 12: Horizontal bar chart of R^2 scores for all five models, with positive values (green) indicating variance explained and negative values (red) indicating models worse than a mean baseline (higher is better).

The R^2 chart provides the starkest illustration of the performance gap between cross-series and per-series models. All four cross-series models (Prophet, Random Forest, Gradient Boosting, Linear Regression) achieve R^2 scores between 0.9656 and 0.9775, meaning they explain over 96.5% of the variance in weekly demand — an exceptional result for a retail forecasting task. ARIMA, by contrast, scores -0.9555 , meaning it performs worse than simply predicting the mean for every observation. This negative R^2 stems from ARIMA fitting 500 independent univariate series without access to cross-series context, store/item identity features, or rolling lag signals, leaving it unable to capture the inter-series demand structure that the Spark MLlib models exploit. Prophet's R^2 of 0.9775 leading all models on this metric (despite its higher RMSE) suggests it captures variance well on typical weeks but occasionally produces large individual errors.

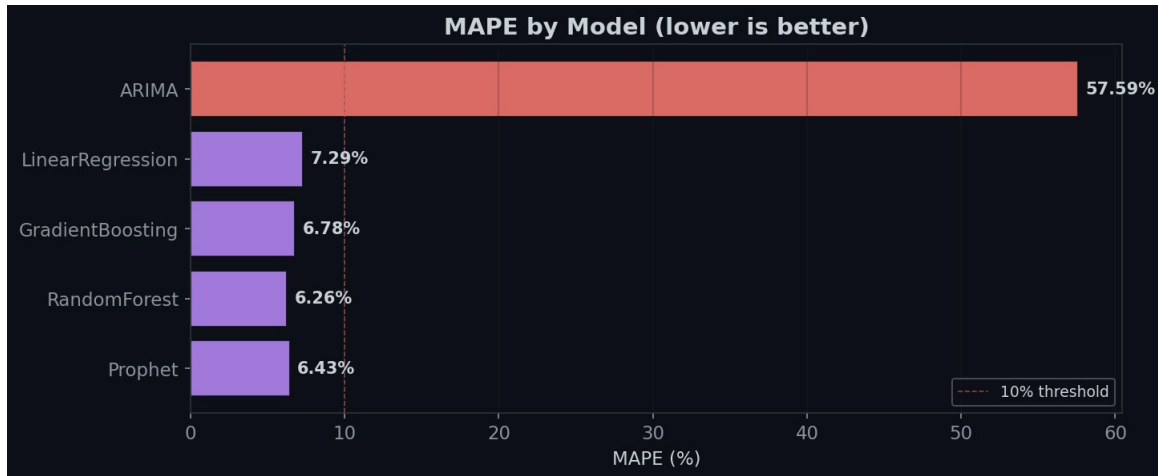
Figure 13 — MAPE by Model

Figure 13: Horizontal bar chart of MAPE (%) for all five models. The dashed red line marks the 10% industry threshold for acceptable forecast accuracy (lower is better).

The MAPE chart delivers the clearest business-interpretable verdict. Random Forest achieves the best MAPE of 6.26%, meaning its forecasts are off by just 6.26% of actual demand on average. Prophet (6.43%), Gradient Boosting (6.78%), and Linear Regression (7.29%) all fall comfortably below the dashed 10% industry-standard threshold for production-quality forecasts. All four models are therefore deployment-ready by conventional retail standards. ARIMA's MAPE of 57.59% is catastrophically high — nearly 9× the 10% threshold — confirming it is not viable for this use case without significant structural modifications (e.g., incorporating exogenous regressors or training on cross-item pooled data). The MAPE ranking (Random Forest > Prophet > Gradient Boosting > Linear Regression) is consistent with the RMSE ranking, reinforcing Random Forest as the recommended production model.